Lecture 8

Pete Manolios
Northeastern

Computer-Aided Reasoning, Lecture 8

SAT, NP-Completeness

- kSAT
  - Literals: variables or their negations
  - Clause: disjunction of literals
  - CNF formula (Conjunctive Normal Form): conjunction of clauses
  - kCNF: CNF formula w/ at most k literals per clause
  - kSAT: The set of satisfiable kCNF formulas
- Recall: SAT (= set of satisfiable propositional formulas) is NP-complete
  - NP: languages whose membership can be verified in P-time
  - NPC: Hardest problems in NP
  - A P-time algorithm for 1 NPC problem implies P-time algorithms for every problem in NP
- 3SAT is NP-complete (Cook 1971)
- SAT $\leq_p$ 3SAT: define P-time function F s.t. $x \in$ SAT iff $F(x) \in$ 3SAT

Tseitin Transformation

- Goal: define a P-time function F s.t. $x \in$ SAT iff $F(x) \in$ 3SAT
- Is there a P-time function F that given g, returns equivalent CNF formula?
  - No: just going from DNF to CNF is *provably* exponential in the worst case
- Idea: define a P-time (linear) function that preserves satisfiability
- Example: $(p \vee (q \wedge \neg r)) \wedge s$
- Introduce definition d: $d = q \wedge \neg r$
- Example is now in CNF: $(p \vee d) \wedge s$
- Turn definition into CNF: $(\neg d \vee q) \wedge (\neg d \vee \neg r) \wedge (\neg q \vee r \vee d)$
- Finally: $(p \vee d) \wedge s \wedge (\neg d \vee q) \wedge (\neg d \vee \neg r) \wedge (\neg q \vee r \vee d)$
- Not equivalent, but *equisatisfiable*
- Using this idea, we recursively introduce definitions as needed

2SAT is in P

- SAT and even 3SAT are NPC. What about 2SAT?
- Construct a graph corresponding to a 2CNF formula F
  - Vertices: $\{x, \neg x : x \text{ a variable}\}$
  - Edges: $\{(u, v), (\neg v, \neg u) : (\neg u \vee v) \in F\}$ $(u, v$ are literals$)$
- Lemma 1: $u \rightarrow v$ (a path from u to v) implies $\neg v \rightarrow \neg u, u \Rightarrow v$
- Lemma 2: F is unsatisfiable iff there is a variable x, st:
  - there is a path from x to $\neg x$, and
  - there is a path from $\neg x$ to x
- Proof: Pong: (A ⇐ B) (lemma 1)
  - Ping: (¬A ⇐ ¬B) (which is equivalent to (A ⇒ B))
  - choose unassigned u s.t. no path to $\neg u$ exists; set u, reachable vertices to true; repeat; notice if $u \rightarrow x, \neg x$, then $\neg x \rightarrow \neg u$, so $u \rightarrow \neg u$
- 2SAT can be solved in linear time
- Run SCC: SAT iff no component contains a literal and its negation

2SAT Example

- $(p \vee q) \wedge (p \vee \neg q) \wedge \neg p$?
- Construct a graph corresponding to a 2CNF formula F
  - Vertices: $\{x, \neg x : x$ a variable$\}$
  - Edges: $\{(u, v), (\neg v, \neg u) : (\neg u \vee v) \in F\}$ (u, v are literals)
- Vertices: $\{p, \neg p, q, \neg q\}$
- Try it!
- Note: for unit clause $\neg p$, treat it as $\neg p \vee \neg p$
- Edges: $\{(\neg p, q), (\neg q, p), (\neg p, \neg q), (q, p), (p, \neg p)\}$
- SCC: $\{\{\neg p, q, p, \neg p\}\}$, so UNSAT

Short History

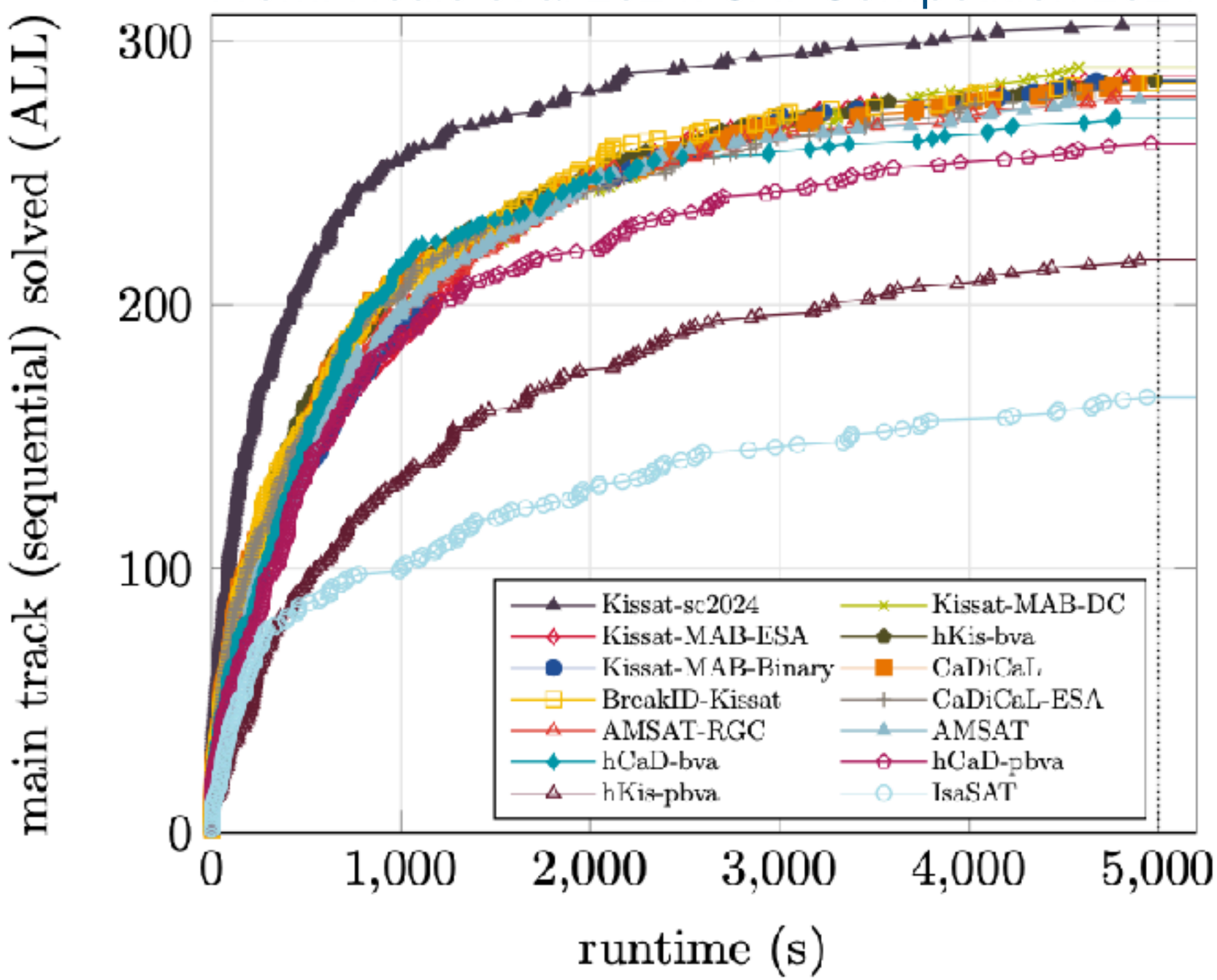
- Ancients: Logic invented as a scientific field of study by Aristotle (380-322 B.C.)
  - Categorical logic, quantifiers, 2-valued, *satisfiability, validity, …*
- Medieval Logicians: early ideas of mechanization, eg, Lull (1232-1315)
- Leibniz (1646-1716): calculus ratiocinator, a kind of calculating machine
- Stanhope (1753-1816): first machine to solve logic problems
- Boole (1815-1864): Boolean algebra
- Frege (1848-1925): Concept notation, basis for modern formal logic
- Russell & Whitehead, Godel, Herbrand, Pierce, Tarski, …
- Shannon (1940): Boolean logic to minimize circuits
- Davis & Putnam (1958): DP algorithm, DPLL (1962)BDDs (Lee 1959), …, ROBDDs (Bryant 1986, …)
- Bryant, Clarke, Emerson & McMillan received the 1998 Paris Kanellakis Award for "their invention of 'symbolic model checking', a method of formally checking system designs widely used in the computer hardware industry."
- CDCL: decision heuristics, backjumping, learning/forgetting, restarts, pre/in-processing, …

SAT Algorithms

- SAT used everywhere: Logic, AI, SAT, scheduling, circuit design, game theory, verification (predicate abstraction), reliability theory, etc.
- Theory vs practice: NPC problem, so what do we do?
  - Option1: give up; not feasible
  - Option2: come up with techniques that work on interesting problems
- If the problem is important, we go with option2
- Multi-billion dollar industries are based on SAT
- An even more extreme example: termination analysis
- When reasoning about systems, SAT is one of the simplest problems we have to deal with

SAT Cactus Plots

From: Heule et al 2024: SAT Competition 2024



DP SAT Algorithm

- Davis Putnam (1960)
- Input: CNF formula
- Output: SAT/UNSAT
- Idea: apply three rules until
  - Derive the empty clause: UNSAT (identity of $\vee$ is false)
  - No clauses remain: SAT (identity of $\wedge$ is true)
- Three “rules”
  - Pure literal rule (affirmative-negative rule)
  - Unit resolution rule (unit propagation, BCP, 1-literal rule)
  - Resolution (Called consensus, also used for logic minimization)

Pure Literal Rule

- Given F, a set of clauses, and literal ℓ such
  - ℓ appears in F
  - $\neg\ell$ does not appear in F
  - remove all clauses containing ℓ
- Equisatisfiable because we can make ℓ true
- Notice that this always simplifies F
- Modern SAT solvers tend to not use the rule (efficiency)

Boolean Constraint Propagation

Unit resolution rule:

$$\frac{C, \neg\ell \qquad \ell}{C}$$

- BCP: given a set of clauses including $\{\ell\}$
  - remove all other clauses containing ℓ (subsumption)
  - remove all occurrences of $\neg\ell$ in clauses (unit resolution)
  - repeat until a fixpoint is reached
- Shannon expansion using conditioning:
  - $f(\ldots,x,\ldots) \equiv [x \wedge f|_x] \vee [\neg x \wedge f|_{\neg x}]$

Resolution

Resolution rule:

$$\frac{C, v \qquad D, \neg v}{C, D} \quad \neg v, v \notin C, D$$

Resolution rule:

$$\frac{C_i, p \qquad D_i, \neg p}{C_i, D_i} \quad \neg p \notin C_i \in P, p \notin D_i \in N$$

- Soundness of rule: above line implies below line
- If below line is SAT, so is above line (w/ side conditions)
- Given literal *p*, set of clauses *S*, let *P* be the clauses in *S* that contain *p* only positively and let *N* be the clauses that contain *p* only negatively. Let *E* be the rest of the clauses. Then *S* is SAT iff *S'* is SAT, where *S'*= *E* U the set of all *p*-resolvents of *P* and *N*.
- Proof: If *A* is an assignment for *S*, then if *A*(*p*)=true, all clauses in *N*, with ¬*p* removed are satisfied, so each *p*-resolvent is satisfied. Similarly if *A*(*p*)=false. If *A* is an assignment for *S'*, then it satisfies all *Ci* or all *Di*: suppose it doesn't satisfy *Ck*, then it must satisfy all *Di*. If it satisfies all *Ci*, let *A'*(*p*)=false, else *A'*(*p*)=true and *A'*(*x*)=*A*(*x*) otherwise.

Resolution Example

Resolution rule:

$$\frac{C, v \qquad D, \neg v}{C, D}$$

C, D are clauses, $\neg v \notin C$ and $v \notin D$

Given literal *p*, set of clauses *S*, let *P* be the clauses in *S* that contain *p* only positively and let *N* be the clauses that contain *p* only negatively. Let *E* be the rest of the clauses. Then *S* is SAT iff *S'* is SAT, where *S'*= *E* U the set of all *p*-resolvents of *P* and *N*.

$$\{\{\neg p, q, r, s\}, \{p, \neg q, s\}, \{\neg p, \neg q, r, \neg s\}, \{p, \neg r, \neg s\}, \{\neg p, \neg q, \neg r\}, \{p, q\}, \{\neg p, \neg q, s\}\}$$

Resolve on *q*

$$\{\{p, \neg r, \neg s\}, \{\neg p, r, s\}, \{p, s\}\}$$

$$\{\neg p, p, r, s\}$$

Notice that clauses that contain a literal and its negation can be thrown away. Why?

Resolution Example

Resolution rule:

$$\frac{C, v \qquad D, \neg v}{C, D}$$ C, D are clauses, $\neg v \notin C$ and $v \notin D$

Given literal *p*, set of clauses *S*, let *P* be the clauses in *S* that contain *p* only positively and let *N* be the clauses that contain *p* only negatively. Let *E* be the rest of the clauses. Then *S* is SAT iff *S'* is SAT, where *S'*= *E* U the set of all *p*-resolvents of *P* and *N*.

$$\{\{\neg p, q, r, s\}, \{p, \neg q, s\}, \{\neg p, \neg q, r, \neg s\}, \{p, \neg r, \neg s\}, \{\neg p, \neg q, \neg r\}, \{p, q\}, \{\neg p, \neg q, s\}\}$$

Resolve on *q* $\{\neg p, p, r, s\}$

$\{\{p, \neg r, \neg s\}, \{\neg p, r, s\}, \{p, s\}\}$

Notice that clauses that contain a literal and its negation can be thrown away. Why?

Resolve on *r*

$\{\{p, s\}\}$ Sat, resolve on *p* to get {} or use pure literal rule

How do we generate a satisfying assignment?